QUESTION 1

What is a primary benefit of using fixed-point arithmetic over floating-point arithmetic in embedded systems?

Not Attempted **Score: 0 / 1**

Options

A. Increased code readability

B. Improved computational accuracy

Expected

C. Reduced power consumption and execution time

FEEDBACK

Explanation: Fixed-point arithmetic is generally faster and less power-intensive than floating-point arithmetic, which is crucial for the resource-constrained environments of embedded systems.

QUESTION 2

When writing embedded system code, what is an important best practice for memory management to improve code readability and maintainability?

Not Attempted **Score: 0 / 1**

Options

A. Using global variables to access data from any part of the program.

B. Favoring dynamic memory allocation for all data structures to provide flexibility.

Expected

C. Utilizing local variables within functions and passing data as arguments for clear data flow.

D. Implementing complex memory management algorithms for optimal memory usage.

FEEDBACK

Explanation: Local variables within functions limit their scope and improve code readability. Passing data as arguments explicitly defines dependencies and data flow between functions. This approach promotes code maintainability and reduces the risk of unintended side effects that can arise from using global variables. Complex memory management algorithms might not be necessary for typical embedded systems and could add unnecessary complexity.

QUESTION 3

Which of the following is a common use case for enum in embedded systems?

Not Attempted **Score: 0 / 1**

Options

Expected

A. To create a set of named integer constants that represent distinct states or modes.

B. To dynamically allocate memory for various data types.

C. To define a structure with multiple data types.

D. To share memory between multiple variables.

FEEDBACK

Explanation: enum is used to define a set of named integer constants, which is particularly useful in embedded systems for representing distinct states, modes, or configuration options in a readable and maintainable way.

QUESTION 4

What does the following code print? Consider the little-Endian memory representation.

```
int x = 65;
char *ptr = (char*)&x;
printf("%c", *ptr);
```

Not Attempted **Score: 0 / 1**

Options

A. 65

Expected

B. A

C. Undefined behavior

D. Error

FEEDBACK

Explanation: The integer x with a value of 65 is typecast to a char pointer. The ASCII value 65 corresponds to the character 'A'. Therefore, *ptr prints 'A'.

QUESTION 5

Which of the following is the recommended and correct way to assign the memory address 0x9ABCDEF0, which stores one byte, to the pointer ptr directly?

Not Attempted **Score: 0 / 1**

Options

A. char ptr = 0x9ABCDEF0;

B. char* ptr = 0x9ABCDEF0;

C. char* ptr = char(0x9ABCDEF0);

Expected

D. char* ptr = (char*)0x9ABCDEF0;

FEEDBACK

Explanation: Option D is correct because it explicitly casts the integer address 0x9ABCDEF0 to a pointer to a char type (char*). This ensures that the pointer ptr properly holds the memory address 0x9ABCDEF0, allowing it to point to the memory location where a single byte can be stored.

QUESTION 6

Which of the following statements correctly increments a float pointer fp by 2 float positions?

Not Attempted **Score: 0 / 1**

Options

Expected

A. fp = fp + 2;

B. fp = (float *)((int)fp + 2);

C. fp = (float *)((char *)fp + 2);

D. fp = (float *)((int)fp + 8);

FEEDBACK

Explanation: Incrementing a float pointer by 2 moves the pointer ahead by 2 times the size of a float. fp = fp + 2; is the correct and simplest way to achieve this.

QUESTION 7

What is the result of 5 & 3?

Not Attempted **Score: 0 / 1**

Options

A. 0

Expected

B. 1

C. 3

D. 2

FEEDBACK

Explanation The & operator performs a bitwise AND. The binary representation of 5 is 0101 and for 3 is 0011. Performing a bitwise AND gives 0001, which is 1 in decimal.

QUESTION 8

What does the expression x |= (1 << n); do? Consider bit position in 0-indexing.

Not Attempted **Score: 0 / 1**

Options

A. Clears the bit number n(from right) of x

B. Toggles the bit number n(from right) of x

Expected

C. Sets the bit number n(from right) of x

D. Inverts all bits from right of x

FEEDBACK

Explanation: The expression (1 << n) creates a mask with only the (n-1)th bit(in 0-indexing) set to 1. The |= operator sets the bit number n-1 of x to 1 without affecting the other bits.

QUESTION 9

What is the result of 0x03 << 2?

Not Attempted **Score: 0 / 1**

Options

A. 0x0F

B. 0x0B

Expected

C. 0x0C

D. 0x0A

FEEDBACK

Explanation: The left shift operation 0x03 << 2 shifts the binary representation of 0x03 (which is 00000011) two positions to the left, resulting in 00001100. This binary value corresponds to 0x0C in hexadecimal.

QUESTION 10

What is a dangling pointer in the context of embedded systems programming?

Answer is Wrong **Score: 0 / 1**

Options

Wrong

A. A pointer that points to a null address

Expected

B. A pointer that points to a memory location which has been freed or is out of scope

C. A pointer that points to a memory location not initialized

D. A pointer that points to a volatile variable

FEEDBACK

Explanation: A dangling pointer is a pointer that points to a memory location that has been freed or whose lifetime has ended, such as a local variable that has gone out of scope. Accessing such pointers can lead to undefined b